

лабораторная работа №7

Имитационное моделирование

Mehmet Efe Kantoz

2026-08-31

Содержание

1	Цель работы	5
2	Задание	6
3	Теоретическое введение	7
3.1	Модель М/М/с	7
3.2	Модель Росса	7
4	Выполнение лабораторной работы	9
4.1	Модель М/М/с: Логика модели	9
4.2	Модель М/М/с: Базовый прогон	9
4.3	Модель М/М/с: Производные форматы	10
4.4	Модель М/М/с: Notebook	10
4.5	Модель Росса: Логика модели	15
4.6	Модель Росса: Прогон модели	16
4.7	Модель Росса: Производные форматы	16
4.8	Модель Росса: Notebook	17
4.9	Модель Росса: Результаты и график	17
5	Выводы	22
	Список литературы	23

Список иллюстраций

4.1	Логика модели	9
4.2	Базовый прогон	10
4.3	Производные форматы	10
4.4	notebook	11
4.5	Модель Росса	16
4.6	Прогон модели	16
4.7	Производные форматы	17
4.8	notebook	17
4.9	График	18

Список таблиц

1 Цель работы

Целью данной лабораторной работы является реализация имитационной модели Росса (задачи о ремонте оборудования) с использованием дискретнособытийного моделирования и анализ показателей надежности системы при различных нагрузках.

2 Задание

- Создать рабочий каталог для кода.
- Установить необходимые пакеты.
- Выполнить предложенный код.
- Преобразовать код в литературный стиль.
- Сгенерировать из литературного кода:
 - чистый код;
 - jupyter notebook;
 - документацию в формате Quarto.
- Выполнить код из jupyter notebook.
- Интегрировать документацию в формате Quarto в отчёт.
- Добавить в код в литературном стиле вычисление для набора параметров.
- Сгенерировать из литературного кода с параметрами:
 - чистый код;
 - jupyter notebook;
 - документацию в формате Quarto.
- Выполнить код из jupyter notebook с параметрами.
- Интегрировать документацию с параметрами в формате Quarto в отчёт.

3 Теоретическое введение

3.1 Модель M/M/c

Модель M/M/c (по классификации Кендалла) — это система массового обслуживания со следующими свойствами: - M (Markovian) — входящий поток заявок пуассоновский, интервалы между прибытиями распределены экспоненциально с параметром λ . - M — время обслуживания каждой заявки распределено экспоненциально с параметром μ . - c — количество идентичных обслуживающих приборов (каналов), работающих параллельно.

3.2 Модель Росса

Модель представляет собой классический пример системы массового обслуживания с конечной популяцией, резервом и ремонтом.

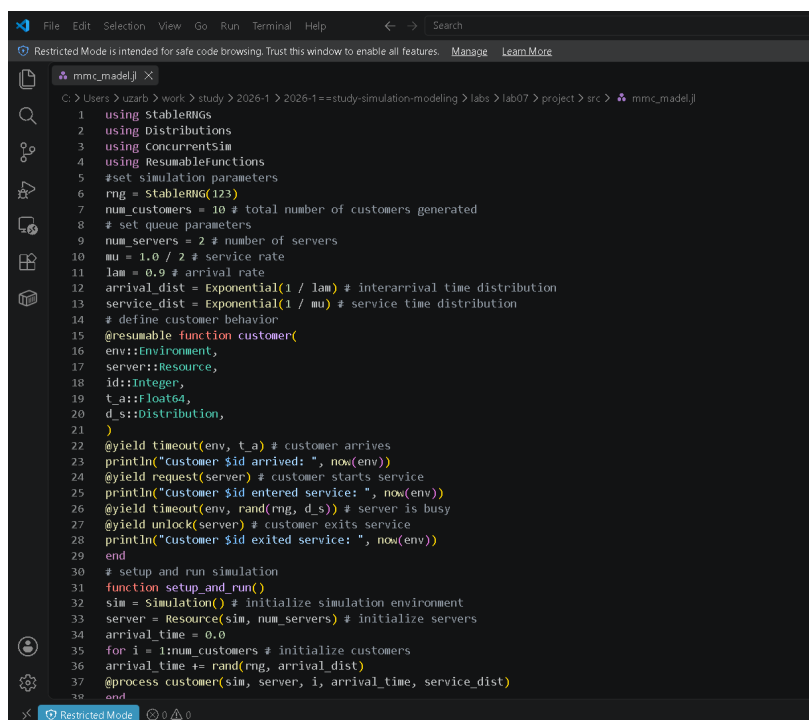
- В системе находятся N идентичных машин, которые постоянно работают и могут выходить из строя.
- S машин находятся в резерве и готовы немедленно заменить любую отказавшую.
- Одно ремонтное устройство (ремонтник), которое может одновременно ремонтировать только одну машину.
- Когда работающая машина ломается, происходит следующее:

- Немедленно берётся одна резервная машина (если она есть) и запускается в работу вместо сломавшейся.
 - Сломанная машина отправляется в ремонт.
 - Если резерва нет, система падает (crash). Моделирование заканчивается.
 - После ремонта машина пополняет пул резервных (становится исправной и ждёт).
- Требуется оценить среднее время до падения системы $E[T]$ при заданных распределениях наработки до отказа и времени ремонта.

4 Выполнение лабораторной работы

4.1 Модель М/М/с: Логика модели

Создаю в папке /src файл с логикой модели (рис. 4.1).



```
1 using Stablenrgs
2 using Distributions
3 using ConcurrentSim
4 using ResumableFunctions
5 #set simulation parameters
6 rng = Stablenrg(123)
7 num_customers = 10 # total number of customers generated
8 # set queue parameters
9 num_servers = 2 # number of servers
10 mu = 1.0 / 2 # service rate
11 lam = 0.9 # arrival rate
12 arrival_dist = Exponential(1 / lam) # interarrival time distribution
13 service_dist = Exponential(1 / mu) # service time distribution
14 # define customer behavior
15 @resumable function customer(
16   env::Environment,
17   server::Resource,
18   id::Integer,
19   t_a::Float64,
20   d_s::Distribution,
21 )
22   @yield timeout(env, t_a) # customer arrives
23   println("customer $id arrived: ", now(env))
24   @yield request(server) # customer starts service
25   println("customer $id entered service: ", now(env))
26   @yield timeout(env, rand(rng, d_s)) # server is busy
27   @yield unlock(server) # customer exits service
28   println("customer $id exited service: ", now(env))
29 end
30 # setup and run simulation
31 function setup_and_run()
32   sim = Simulation() # initialize simulation environment
33   server = Resource(sim, num_servers) # initialize servers
34   arrival_time = 0.0
35   for i = 1:num_customers # initialize customers
36     arrival_time += rand(rng, arrival_dist)
37   @process customer(sim, server, i, arrival_time, service_dist)
38 end
```

Рисунок 4.1: Логика модели

4.2 Модель М/М/с: Базовый прогон

Далее создаю и запускаю код с базовым прогоном модели (рис. 4.2).

```
mmc-basic.jl:43
PS C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab07\project> julia scripts/mmc-basic.jl
Activating project at 'C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab07\project'
Моделирование завершено. Графики сохранены в plots/, данные в data/.
PS C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab07\project> |
gqlentuu..
```

Рисунок 4.2: Базовый прогон

4.3 Модель М/М/с: Производные форматы

Создаю все производные форматы (рис. 4.3).

```
Моделирование завершено. Графики сохранены в plots/, данные в data/.
PS C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab07\project> julia scripts/tangle.jl script
s/mmc-basic.jl
Activating project at 'C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab07\project'
Генерация из: scripts/mmc-basic.jl
[ Info: generating plain script file from 'C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab07
\project\scripts\mmc-basic.jl'
[ Info: writing result to 'C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab07\project\scripts
\mmc-basic\mmc-basic.jl'
✓ Чистый скрипт: C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab07\project\scripts\mmc-basi
c\mmc-basic.jl
[ Info: generating markdown page from 'C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab07\pro
ject\scripts\mmc-basic.jl'
[ Info: writing result to 'C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab07\project\markdow
n\mmc-basic\mmc-basic.qmd'
✓ Quarto: C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab07\project\markdown\mmc-basic\mmc-
basic.qmd
[ Info: generating notebook from 'C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab07\project\
scripts\mmc-basic.jl'
[ Info: writing result to 'C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab07\project\noteboo
ks\mmc-basic\mmc-basic.ipynb'
✓ Notebook: C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab07\project\notebooks\mmc-basic\m
mc-basic.ipynb
Готово! Все файлы созданы.
PS C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab07\project> |
```

Рисунок 4.3: Производные форматы

4.4 Модель М/М/с: Notebook

Запускаю файл notebook (рис. 4.4).

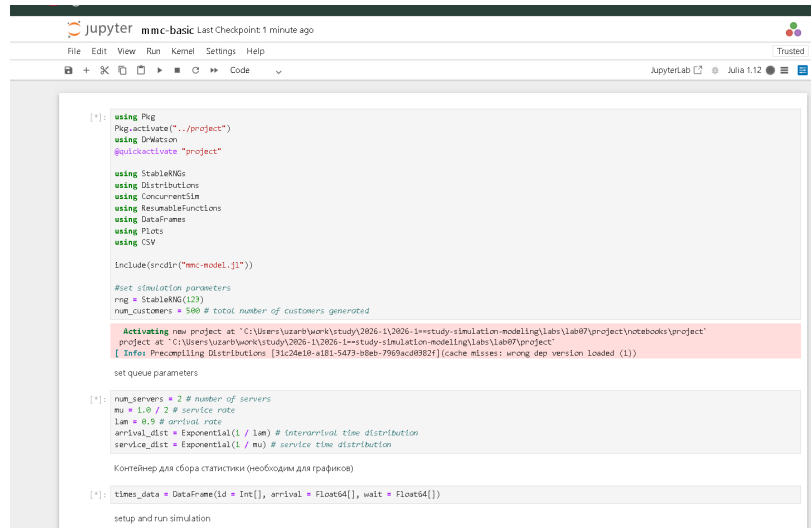


Рисунок 4.4: notebook

```

using Pkg
Pkg.activate("../project")
using DrWatson
@quickactivate "project"

using StableRNGs
using Distributions
using ConcurrentSim
using ResumableFunctions
using DataFrames
using Plots
using CSV

include(scrdir("mmc-model.jl"))

#set simulation parameters
  
```

```
rng = StableRNG(123)
num_customers = 500 # total number of customers generated
```

Activating project at `C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab07\project`

500

set queue parameters

```
num_servers = 2 # number of servers
mu = 1.0 / 2 # service rate
lam = 0.9 # arrival rate
arrival_dist = Exponential(1 / lam) # interarrival time distribution
service_dist = Exponential(1 / mu) # service time distribution
```

Distributions.Exponential{Float64}(λ=2.0)

Контейнер для сбора статистики (необходим для графиков)

```
times_data = DataFrame(id = Int[], arrival = Float64[], wait = Float64[])
```

	id	arrival	wait
	Int64	Float64	Float64

setup and run simulation

```
function setup_and_run()
    sim = Simulation() # initialize simulation environment
    server = Resource(sim, num_servers) # initialize servers
    arrival_time = 0.0
    for i = 1:num_customers # initialize customers
        arrival_time += rand(rng, arrival_dist)
```

```

        @process customer(sim, server, i, arrival_time, service_dist, rn
    end
    run(sim) # run simulation
end

```

setup_and_run (generic function with 1 method)

Выполнение симуляции

```

setup_and_run()

CSV.write(datadir("sim_results_mmc.csv"), times_data)

```

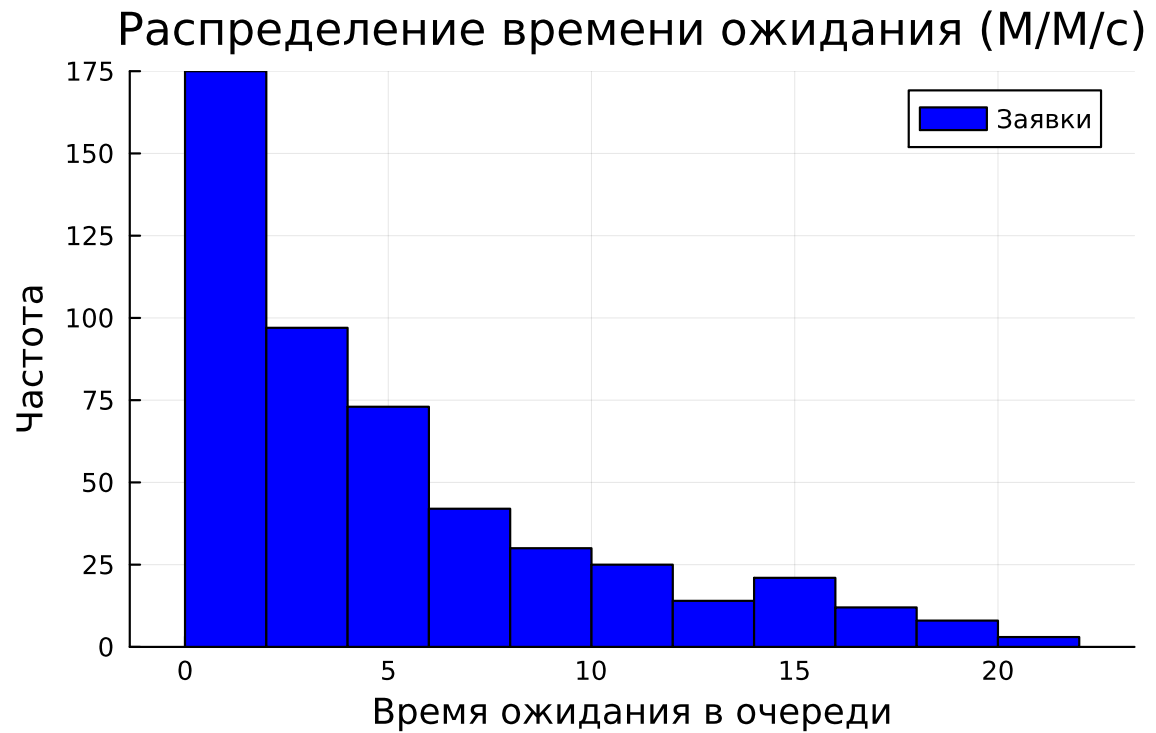
"C:\\Users\\uzarb\\work\\study\\2026-1\\2026-1==study-simulation-modeling\\labs\\lab07\\project\\data\\sim_results_mmc.csv"

Распределение времени ожидания

```

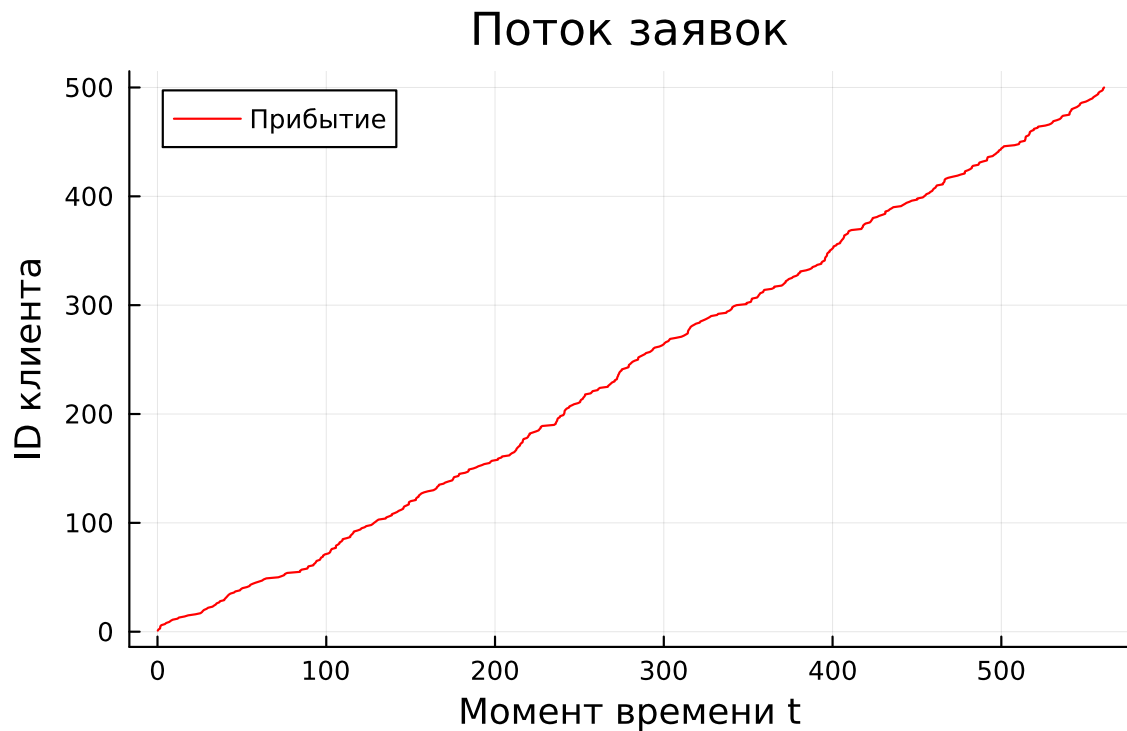
p1 = histogram(times_data.wait,
               title = "Histogram of waiting times (M/M/c)",
               xlabel = "Waiting time (M/M/c)",
               ylabel = "Frequency",
               label = "Frequency",
               color = :blue,
               fmt = :png
            )

```



Динамика прибытия событий

```
p2 = plot(times_data.arrival, times_data.id,
          title = "XXXXX XXXXX",
          xlabel = "XXXXXX XXXXXX t",
          ylabel = "ID XXXXXX",
          label = "XXXXXXXX",
          color = :red,
          fmt = :png
        )
```



Объединение и сохранение графиков

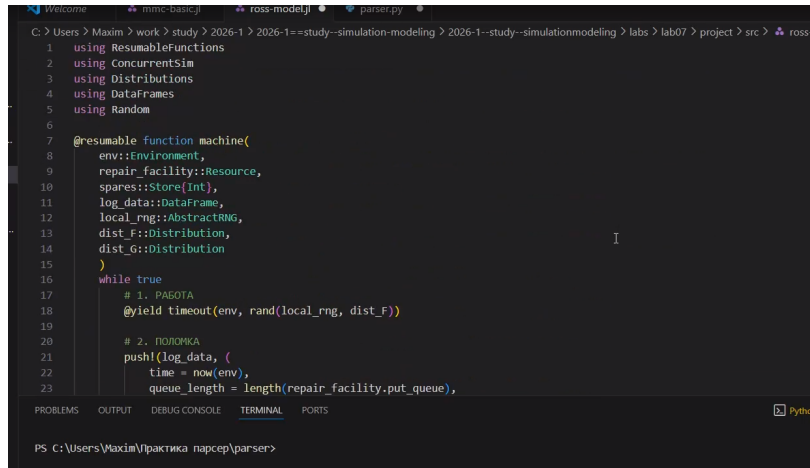
```
final_plot = plot(p1, p2, layout = (2, 1), size = (800, 600))
savefig(plotsdir("mmc_analysis.png"))

println("XXXXXXXXXXXXXXXX XXXXXXXXXXXX. XXXXXXXX XXXXXXXXXXXX ✕ plots/, XXXXXXXX ✕ data/.")
```

```
XXXXXXXXXXXXXXXX XXXXXXXXXXXX. XXXXXXXX XXXXXXXXXXXX ✕ plots/, XXXXXXXX ✕ data/.
```

4.5 Модель Росса: Логика модели

Создаю в папке /src файл с логикой модели Росса (рис. 4.5).

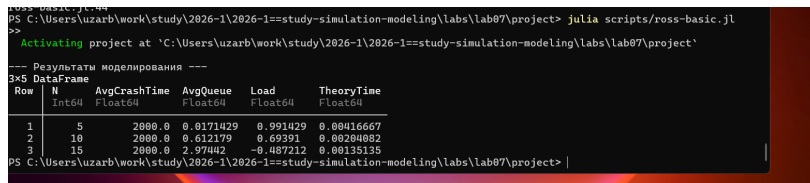


```
C:\Users\Maxim> work > study > 2026-1 > 2026-1==study--simulation-modeling > 2026-1--study--simulationmodeling > labs > lab07 > project > src > ross-model.jl
1 using ResumableFunctions
2 using ConcurrentSim
3 using Distributions
4 using DataFrames
5 using Random
6
7 @resumable function machine(
8     env::Environment,
9     repair_facility::Resource,
10    spares::Store{Int},
11    log_data::DataFrame,
12    local_rng::AbstractRNG,
13    dist_F::Distribution,
14    dist_G::Distribution
15 )
16     while true
17         # 1. РАБОТА
18         @yield timeout(env, rand(local_rng, dist_F))
19
20         # 2. ПОРЧКА
21         push!(log_data, (
22             time = now(env),
23             queue_length = length(repair_facility.put_queue),
24         ))
25     end
26 end
```

Рисунок 4.5: Модель Росса

4.6 Модель Росса: Прогон модели

Запускаю файл с прогоном модели (рис. 4.6).



```
PS C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab07\project> julia scripts/ross-basic.jl
Activating project at 'C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab07\project'

--- Результаты моделирования ---
3x5 DataFrame
Row N AvgCrashTime AvgQueue Load TheoryTime
   Int64 Float64 Float64 Float64 Float64
1 5 2888.0 0.0171429 0.991429 0.00416667
2 10 2888.0 0.612179 0.69391 0.00284882
3 15 2888.0 2.97442 -0.487212 0.00135135
PS C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab07\project>
```

Рисунок 4.6: Прогон модели

4.7 Модель Росса: Производные форматы

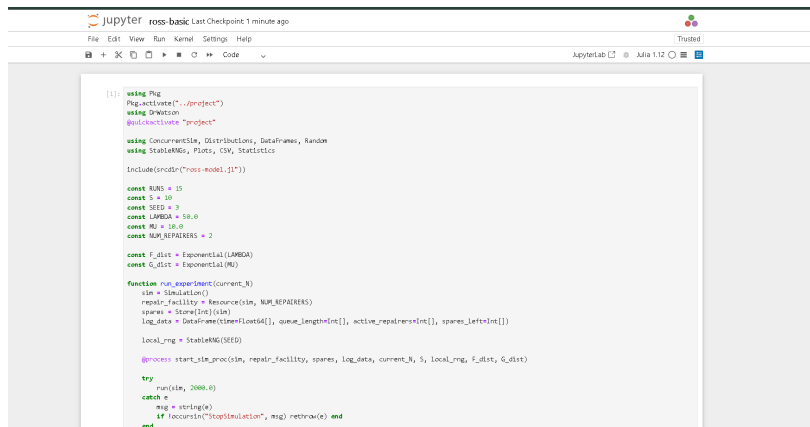
Создаю все производные форматы (рис. 4.7).

```
PS C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab07\project> julia scripts/tangle.jl script
s/ross-basic.jl
Activating project at 'C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab07\project'
Генерация на: scripts/ross-basic.jl
[ Info: generating plain script file from 'C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab07\
\project\scripts\ross-basic.jl'
[ Info: writing result to 'C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab07\project\scripts
\ross-basic\ross-basic.jl'
✓ Notebook output: C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab07\project\scripts\ross-bas
ic\ross-basic.jl
[ Info: generating markdown page from 'C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab07\pro
ject\scripts\ross-basic.jl'
[ Info: writing result to 'C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab07\project\markdow
n\ross-basic\ross-basic.qmd'
✓ Quarto: C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab07\project\markdown\ross-basic\ros
s-basic.qmd
[ Info: generating notebook from 'C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab07\project\
scripts\ross-basic.jl'
[ Info: writing result to 'C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab07\project\noteboo
ks\ross-basic\ross-basic.ipynb'
✓ Notebook: C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab07\project\notebooks\ross-basic\
ross-basic.ipynb
Готово! Все файлы созданы.
PS C:\Users\uzarb\work\study\2026-1\2026-1==study-simulation-modeling\labs\lab07\project> |
```

Рисунок 4.7: Производные форматы

4.8 Модель Росса: Notebook

Запускаю файл notebook (рис. 4.8).



```
[1]: using Plots
      Plots.activate("../project")
      using Distributions
      @docactivate "project"

      using ConcurrentData, Distributions, DataFrames, Random
      using StableRNGs, Plots, CSV, Statistics

      include(joinpath("ross-model.jl"))

      const RUNS = 15
      const S = 10
      const SEED = 3
      const LAMBDA = 58.0
      const MU = 15.0
      const NUM_REPAIRERS = 2

      const F_dist = Exponential(LAMBDA)
      const G_dist = Exponential(MU)

      function run_experiment(current_N)
          sim = Simulation()
          repair_facility = Resource{sin, NUM_REPAIRERS}
          spares = Store{Int}(sin)
          log_data = DataFrame{Vector{Int64}, Queue{length{Int}}, active_repairers{Int}, spares_left{Int}}()

          local_rng = StableRNG(SEED)

          @process start_sim_proc(sim, repair_facility, spares, log_data, current_N, S, local_rng, F_dist, G_dist)

          try
              run(sim, 2000.0)
          catch e
              msg = string(e)
              if occursin("stopSimulation", msg) rethrow(e) end
          end
      end
```

Рисунок 4.8: notebook

4.9 Модель Росса: Результаты и график

В результате получаю следующий график (рис. 4.9).

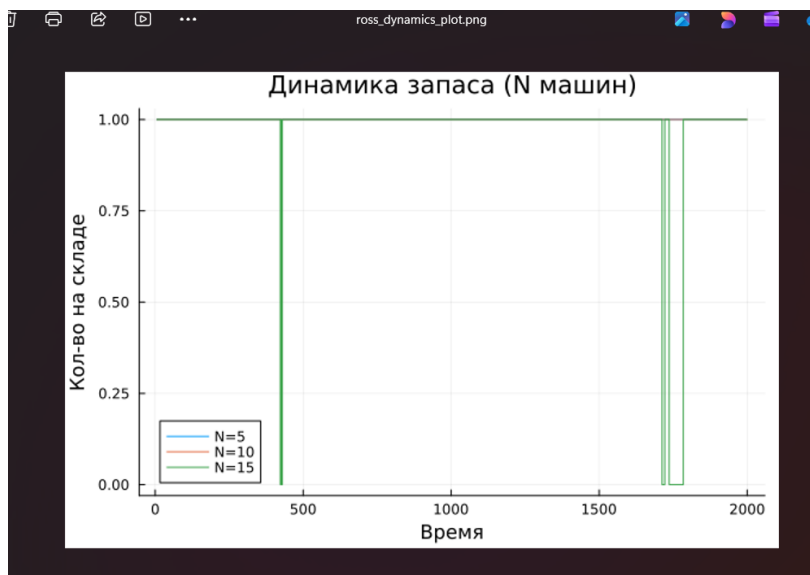


Рисунок 4.9: График

```
using Pkg
Pkg.activate("../project")
using DrWatson
@quickactivate "project"

using ConcurrentSim, Distributions, DataFrames, Random
using StableRNGs, Plots, CSV, Statistics

include(sourcedir("ross-model.jl"))

const RUNS = 15
const S = 10
const SEED = 3
const LAMBDA = 50.0
const MU = 10.0
```

```

const NUM_REPAIRERS = 2

const F_dist = Exponential(LAMBDA)
const G_dist = Exponential(MU)

function run_experiment(current_N)
    sim = Simulation()
    repair_facility = Resource(sim, NUM_REPAIRERS)
    spares = Store{Int}(sim)
    log_data = DataFrame(time=Float64[], queue_length=Int[], active_re

    local_rng = StableRNG(SEED)

    @process start_sim_proc(sim, repair_facility, spares, log_data, cu

    try
        run(sim, 2000.0)
    catch e
        msg = string(e)
        if !occursin("StopSimulation", msg) rethrow(e) end
    end

    return log_data, now(sim)
end

summary_df = DataFrame(N=Int[], AvgCrashTime=Float64[], AvgQueue=Float64[])
p_spares = plot(title="XXXXXXXX XXXXXX (N XXXXX)", xlabel="XXXXX", ylabel="

```


	N	AvgCrashTime	AvgQueue	Load	TheoryTime
	Int64	Float64	Float64	Float64	Float64
1	5	2000.0	0.0171429	0.991429	0.00416667
2	10	2000.0	0.612179	0.69391	0.00204082
3	15	2000.0	2.97442	-0.487212	0.00135135

Сохранение

```
mkpath(datadir("exp_pro"))
CSV.write(datadir("exp_pro", "ross_final_metrics.csv"), summary_df)
savefig(p_spares, plotsdir("ross_dynamics_plot.png"))
```

```
"C:\\Users\\uzarb\\work\\study\\2026-1\\2026-1==study-
simulation-modeling\\labs\\lab07\\project\\plots\\ross_dynamics_plo
```

5 Выводы

В результате выполнения данной лабораторной работы мы научились создавать модель Росса с помощью дискретнособытийного моделирования и анализ показателей надежности системы.

Список литературы